

TECHNICAL PROJECT REPORT

Flight Controller Requirement Standardization System

*Offline Requirement Analysis, Compliance Gating and Controlled
Candidate Generation*

Project Type	Offline Requirements Engineering System
Core Processing	Deterministic Rule Checking + Local Candidate Generation
Deployment Context	Air-gapped / Offline Environment
Project Scope	Flight Controller Requirement Workbooks

Prepared for Technical Review

1. Introduction

Flight controller development depends on requirements that are precise, testable, unambiguous and consistently written. In practice, requirement workbooks often contain statements written by different authors over long development cycles. This creates variation in wording, structure and quality. A requirement may express the correct engineering intent while still containing vague language, multiple thoughts, open-ended clauses, missing measurable criteria or inconsistent terminology.

1.1 Problem Statement

Manual review of large requirement workbooks is time-consuming and difficult to scale consistently. The intended deployment environment also requires offline operation, which prevents dependence on cloud-hosted processing or external network services. The system therefore needs to analyse requirements locally, identify deterministic defects quickly, prevent unnecessary expensive generation, and present reviewable outputs to an engineer.

1.2 Existing Solutions and Gap

- Manual requirement review provides engineering judgement but becomes slow and inconsistent for hundreds of statements.
- Rule-only validation is fast and transparent, but cannot always produce a well-phrased alternative requirement.
- Generation-first approaches can improve wording, but running generation for every requirement wastes compute and can process statements that should have been rejected earlier.

1.3 Solution Explored

The explored solution is an offline requirement standardization pipeline. It combines deterministic checks with local candidate generation. The deterministic path performs inexpensive structural and quality screening first. Only requirements that clear the required gates proceed to local candidate generation through Ollama. Generated candidates are subsequently scored and recommended, while the final review remains visible to a human reviewer.

Objective	How the System Addresses It
Offline operation	All processing is designed for local execution without external network calls.
Fast early screening	Deterministic rules run before candidate generation.
Controlled generation	Local generation is invoked only after requirements clear quality gates.
Traceability	Per-stage progress, rejection reasons, scores and timings are recorded.
Human oversight	Outputs remain available for review rather than being silently accepted.

2. Methodology

2.1 Solution Approach

The methodology separates processing into two distinct execution paths: a deterministic validation path and a local generation path. This separation is important because deterministic checks are extremely fast and explainable, whereas local generation is comparatively expensive on CPU-only hardware.

The pipeline first parses each requirement and applies a sequence of deterministic checks. A requirement that fails a gate is rejected immediately and does not continue to later processing. A requirement that passes all required gates is sent to the local generation stage, where multiple rewrite candidates are produced. The candidates are then evaluated and ranked using deterministic scoring.

2.2 Processing Pipeline

Stage	Purpose	Execution Type
1. Parse	Extract and normalize requirement text from the workbook.	Deterministic
2. RuleFlag	Detect immediately identifiable requirement defects.	Deterministic
3. ClassifyPattern	Classify the requirement against supported structural patterns.	Deterministic
4. AbbreviationCheck	Check acronym and abbreviation issues covered by R37/R38.	Deterministic
5. IncoseCheck	Apply the pre-generation INCOSE quality gate.	Deterministic
6. ComplianceCheck	Validate EARS-related structural compliance.	Deterministic
7. GenerateCandidates	Generate controlled rewrite candidates locally.	Local model
8. Score / Recommend	Score candidates and recommend the best result.	Deterministic
9. Review / Finalize	Expose result for review and export.	Human-in-the-loop

2.3 Assumptions

- The input workbook contains identifiable requirement statements that can be parsed into individual records.
- The deployment machine can run the configured local Ollama model.
- A human reviewer remains responsible for accepting or rejecting the final engineering requirement.
- The INCOSE and EARS checks cover the automatable rules implemented in the project; they do not replace full expert judgement.

3. Requirements Checker Suite

3.1 Deterministic Checking

Deterministic checks are the first line of processing. They do not require generation and therefore provide very low latency. They also produce explainable outcomes because every detected issue can be associated with a rule, condition or structural reason.

- RuleFlag detects configured requirement defects before generation.
- ClassifyPattern determines the relevant EARS-style pattern classification.
- AbbreviationCheck handles acronym and abbreviation checks corresponding to R37 and R38.
- IncoseCheck applies the broader automatable INCOSE rule set before generation.
- ComplianceCheck validates the structural EARS gate before candidate generation.

3.2 Local Candidate Generation

Candidate generation is intentionally placed after deterministic gating. The local model is used to rewrite a requirement into multiple candidate forms rather than to perform all validation. This preserves the role of deterministic rules as the authoritative fast path while using generation only where it adds value.

The system is configured to generate multiple candidates for an eligible requirement. These candidates are subsequently evaluated using deterministic scoring and recommendation logic. This design avoids compromising candidate generation quality while reducing unnecessary model calls for clearly unsuitable inputs.

3.3 Why the Split is Important

Characteristic	Deterministic Path	Local Generation Path
Speed	Very low latency; typically sub-millisecond for the complete rule path.	Seconds to minutes depending on hardware and input.
Explainability	Direct rule IDs and specific failure reasons.	Candidate wording requires downstream scoring and review.
Compute cost	Low CPU cost.	Dominant compute cost on CPU-only deployment.
Best use	Filtering, checking, scoring and rejection.	Producing alternative candidate wording.

4. Pre-Generation INCOSE Compliance Gate

4.1 Motivation for the Change

Previously, the pipeline relied on the EARS structure check as the principal deterministic gate before local generation. Broader INCOSE checking was primarily used after candidates had already been generated, where it contributed to candidate scoring and ranking. This meant a requirement could be structurally acceptable but still contain several quality defects and nevertheless consume local model time.

The updated design introduces a dedicated IncoseCheck gate before the EARS compliance gate. The objective is not to reduce the quality or number of candidates for valid requirements; instead, it prevents clearly non-compliant inputs from reaching the expensive generation stage.

4.2 Pipeline Order

Previous Flow	Updated Flow
Parse → RuleFlag → ClassifyPattern → AbbreviationCheck → ComplianceCheck → GenerateCandidates	Parse → RuleFlag → ClassifyPattern → AbbreviationCheck → IncoseCheck → ComplianceCheck → GenerateCandidates

4.3 Gate Behaviour

The IncoseCheck runs the deterministic INCOSE scorer against the original requirement text. It excludes R37 and R38 because those checks have already been performed by AbbreviationCheck. R1 is also excluded because structured-statement validation is handled by the EARS-related compliance gate immediately after IncoseCheck. This avoids duplicated checks and avoids presenting the reviewer with redundant rejection reasons.

The gate produces a score out of 100 and compares it with pipeline.incose_gate_threshold. The configured default threshold is 80.0. Requirements below this threshold are rejected immediately; the EARS gate and local generation are skipped.

5. Test Rules and Test Data

5.1 INCOSE Rule Evaluation

The INCOSE scorer evaluates the implemented automatable rule set and can return both an overall score and per-rule findings. The pre-generation gate uses these findings to determine whether a requirement should continue. The design deliberately preserves specific rule information rather than returning only a numeric score.

5.2 Detailed Rejection Reasons

A failed requirement now receives an explanation containing the overall score, threshold and the individual failed rule details. For example, a rejection can identify vague terms, open-ended clauses, multiple 'shall' statements, combinators, absolutes, universal qualification problems or missing measurable performance criteria. Each finding is associated with the rule identifier, rule title and specific reason generated by the deterministic checker.

5.3 EARS Structural Validation

After a requirement clears the INCOSE gate, the EARS compliance check validates whether the requirement satisfies the expected structural pattern. A failure at this stage also stops further processing. Therefore, local candidate generation is reserved for requirements that have passed both the broader INCOSE quality gate and the structural EARS gate.

5.4 Test Data and Validation

The updated gate was evaluated using the project's two golden datasets: data/golden/eval.json and data/golden/fewshot.json. A total of 210 real examples were re-run through the full gated pipeline. At the configured threshold of 80.0, the validation reported 0 false rejections in these datasets.

Validation Item	Observed Result
Golden dataset examples	210 examples evaluated
INCOSE gate threshold	80.0 / 100
False rejections in validation run	0
Full offline test suite	460 passing tests
New tests added	12 tests covering gate and UI behaviour

6. Test Execution and Result Scenarios

6.1 Pass Scenario

A passing requirement proceeds through parsing, deterministic rule checks, pattern classification, abbreviation validation, INCOSE evaluation and EARS compliance. Once both gates pass, local candidate generation is executed. The generated candidates are then deterministically scored and a recommended candidate is produced for review.

Stage	Pass Behaviour
IncaseCheck	Score meets or exceeds the configured threshold.
ComplianceCheck	Requirement satisfies the EARS structural gate.
GenerateCandidates	Configured candidate rewrites are generated locally.
Score / Recommend	Candidates are evaluated and the strongest candidate is recommended.
Finalize	Result is made available for export or human review.

6.2 Fail Scenario: INCOSE Gate

If the INCOSE score is below the threshold, the requirement is rejected immediately. The EARS gate is skipped and no local generation call is made. The rejection output includes the score, threshold and detailed failed-rule reasons.

6.3 Fail Scenario: EARS Gate

If the requirement passes IncaseCheck but fails the EARS compliance gate, generation is also skipped. The output records the structural compliance reason, allowing the reviewer to understand why the requirement did not qualify for candidate generation.

6.4 Failure Behaviour Summary

Failure Point	System Behaviour	Recorded Output
INCOSE Gate	Reject immediately; later gate and generation are skipped.	Score, threshold, failed rule IDs, titles and reasons.
EARS Gate	Reject after INCOSE pass; generation is skipped.	Structural compliance result and EARS-related reason.
Generation / Review	Executed only after deterministic gates pass.	Candidate outputs, scores and recommendation.

7. Results and Performance Observations

7.1 Deterministic Path Performance

The execution console makes the difference between deterministic processing and local generation visible. The deterministic path, including rule checks, EARS-related validation and the pre-generation INCOSE evaluation, completes in well under a millisecond in the observed CPU-only setup. The console can record this using messages such as '[all deterministic checks done in 0.62 ms]'.

7.2 Local Generation Performance

Local candidate generation is the dominant source of processing time. On the CPU-only deployment using gemma3:4b, generation can take from seconds to several minutes depending on requirement complexity, prompt size, model execution and available hardware. The console therefore records the actual wall-clock time for generation, for example: '3 candidate rewrite(s) generated via Ollama in 42.3s'.

7.3 Result of the New Gate

The new gate improves overall efficiency by reducing wasted model invocations. A requirement that contains several deterministic quality defects can now be rejected before generation. Importantly, candidate generation for requirements that pass both gates is unchanged: eligible requirements still reach the same generation and downstream scoring process.

Processing Area	Observed Behaviour
Deterministic checks	Fast and explainable; negligible compared with generation time.
Invalid requirements	Can be rejected before model invocation.
Valid requirements	Continue to local multi-candidate generation.
Console visibility	Per-stage pass/fail, score, failure details and timings are available.
Scalability benefit	Avoids spending generation time on requirements that fail objective pre-checks.

8. Inference and Discussion

8.1 Main Inference

The principal performance bottleneck is not the deterministic rule engine. The measured architecture shows that deterministic processing is extremely fast, while local model inference dominates total execution time. Therefore, meaningful improvement for large workbooks is achieved primarily by reducing unnecessary model calls and optimising model execution, rather than attempting to micro-optimize the deterministic rule code.

8.2 Why the Gate Does Not Compromise Candidate Generation

The design does not reduce the number of candidates for requirements that are eligible for generation. Instead, it changes which requirements are permitted to consume generation resources. A requirement must first demonstrate sufficient deterministic quality. Once it clears the gates, the configured generation process remains available in full.

8.3 Practical Impact for Large Workbooks

For a workbook containing hundreds of requirements, processing every statement through a local model sequentially can produce substantial execution time. The pre-generation gate improves the situation whenever a meaningful portion of the workbook contains deterministically detectable defects. The total time saved is approximately proportional to the number of model calls avoided.

8.4 Explainability and Review Value

Detailed rejection reasons improve reviewability. A reviewer no longer sees only a low score; the system reports the rules that failed and the specific reasons. This is particularly useful in a requirements engineering workflow because it turns a rejection into actionable feedback rather than an unexplained system decision.

9. Limitations and Constraints

9.1 Local Hardware Constraint

The current deployment is CPU-bound for local generation. The deterministic path is not the limiting factor; model inference is. As a result, generation time can vary significantly with processor capability, memory availability, model size and requirement complexity.

9.2 Rule Coverage Constraint

Only rules that are suitable for deterministic automation are enforced automatically. Some aspects of requirement quality remain context-dependent and require engineering judgement. The system therefore supports review rather than replacing the reviewer.

9.3 Threshold Sensitivity

The INCOSE gate threshold is configurable. A stricter threshold can reject more inputs and save more generation time, but may also increase the risk of rejecting requirements that could have benefited from rewriting. The selected default of 80.0 was validated against the available 210-example golden dataset with 0 observed false rejections.

9.4 Sequential Processing Constraint

If the current pipeline processes requirements sequentially, very large workbooks will still inherit the accumulated latency of each local generation call. Future optimisation must consider safe concurrency, batching or hardware acceleration while preserving memory limits, deterministic traceability and controlled review behaviour.

10. Future Work

10.1 Performance Improvements

- Introduce controlled parallelism for independent deterministic operations and, where safe, carefully managed generation requests.
- Evaluate smaller or quantized local models that preserve rewrite quality while reducing inference latency.
- Use hardware acceleration where the deployment environment permits it.
- Profile token generation, prompt construction and model startup overhead separately to identify the dominant generation sub-stage.
- Cache repeated or identical requirement processing where traceability and version control allow it.

10.2 Workflow Improvements

- Provide a pre-processing summary showing how many requirements passed, failed and reached generation.
- Support batch-level estimates of expected processing time based on observed generation latency.
- Expose rule-failure trends across the workbook to help engineers identify recurring authoring problems.
- Expand export reporting with requirement IDs, gate outcomes, detailed rejection reasons, candidates, scores and reviewer decisions.

10.3 Conclusion

The Flight Controller Requirement Standardization System demonstrates an offline approach to scalable requirement analysis by combining deterministic validation with controlled local candidate generation. The addition of the pre-generation INCOSE compliance gate strengthens the pipeline by rejecting unsuitable requirements before they consume expensive model time. Validation on the available 210-example golden dataset reported no false rejections at the configured threshold, while the full offline test suite reported 460 passing tests. The resulting architecture makes the performance boundary explicit: deterministic checking is fast, while local generation is the primary latency source. This provides a clear direction for future optimisation without weakening the quality controls or candidate generation workflow.

End of Report